

**a) Problem: Cross Correlation Analysis of variable input signals:**

$x1=[1, 2, 4, 7, 12]$

$x2=[0, 1, 1, 2, 6, 9, 14]$

**Logical Solution:**

- $$r_{xy}(n) = \sum_{k=-\infty}^{\infty} x(k) * y(k - n)$$

**Source Code:**

```
def correlation(x,x0,y,y0):
    t=[]
    rxy=[]
    mxx=-100000
    # rxy(n)= sum(k=-inf to inf) x(k)*y(k-n)
    mx=max(len(x),len(y))
    for n in range(-mx,mx+1):
        rxy.append(0)
        t.append(n)
        for k in range(-mx,mx+1):
            try:                # if positive index exist?
                if k>=(-x0): a=x[k+x0] # if negative index exists?
            else: a=0
            except: a=0

            try:
                if (k-n)>=(-y0): b=y[k+y0-n]
            else: b=0
            except: b=0

            rxy[-1]+=a*b

        if mxx<=rxy[-1]:
            delay=t[-1]
            mxx=rxy[-1]

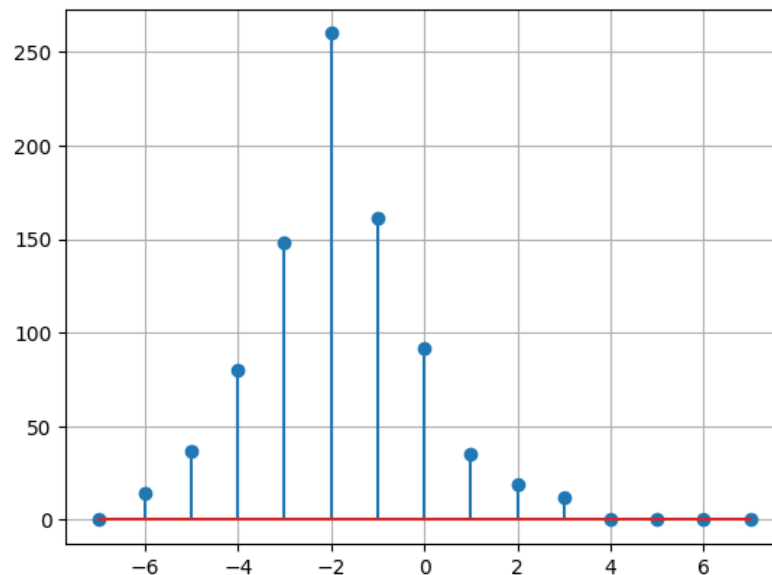
    return t,rxy,(delay,mxx)

x1=[1,2,4,7,12]
x2=[0,1,1,2,6,9,14]
```

```
t, rxy, result = correlation(x1, 0, x2, 0)
print("Time Sequence:", t)
print("Correlation Values:", rxy)
print("Delay:", result[0])
print("Maximum Value:", result[1])
```

```
Time Sequence: [-7, -6, -5, -4, -3, -2, -1, 0, 1, 2, 3, 4, 5, 6, 7]
Correlation Values: [0, 14, 37, 80, 148, 260, 161, 92, 35, 19, 12, 0, 0, 0, 0]
Delay: -2
Maximum Value: 260
```

```
plt.stem(t, rxy)
plt.grid(True)
plt.show()
```



*Figure-Correlated Signal*

### Discussion:

- In signal processing, when two signals are correlated, it means they have mathematical **similarity** to each other.
- It represents how much two signals similar each other as one slide past the other in time.

## b) Problem: Cross Correlation of Sine and Shifted Sine Signal

### Logical Solution:

- $$r_{xy}(n) = \sum_{k=-\infty}^{\infty} x(k) * y(k - n)$$

### Source Code:

```
import numpy as np
import matplotlib.pyplot as plt

f=1
t=np.linspace(-1,1,100)
sinx=np.sin(2*np.pi*f*t)

shifted_sign=list(sinx)
t=5
for i in range(t):
    shifted_sign.insert(0,0)

shifted_sign=np.array(shifted_sign)

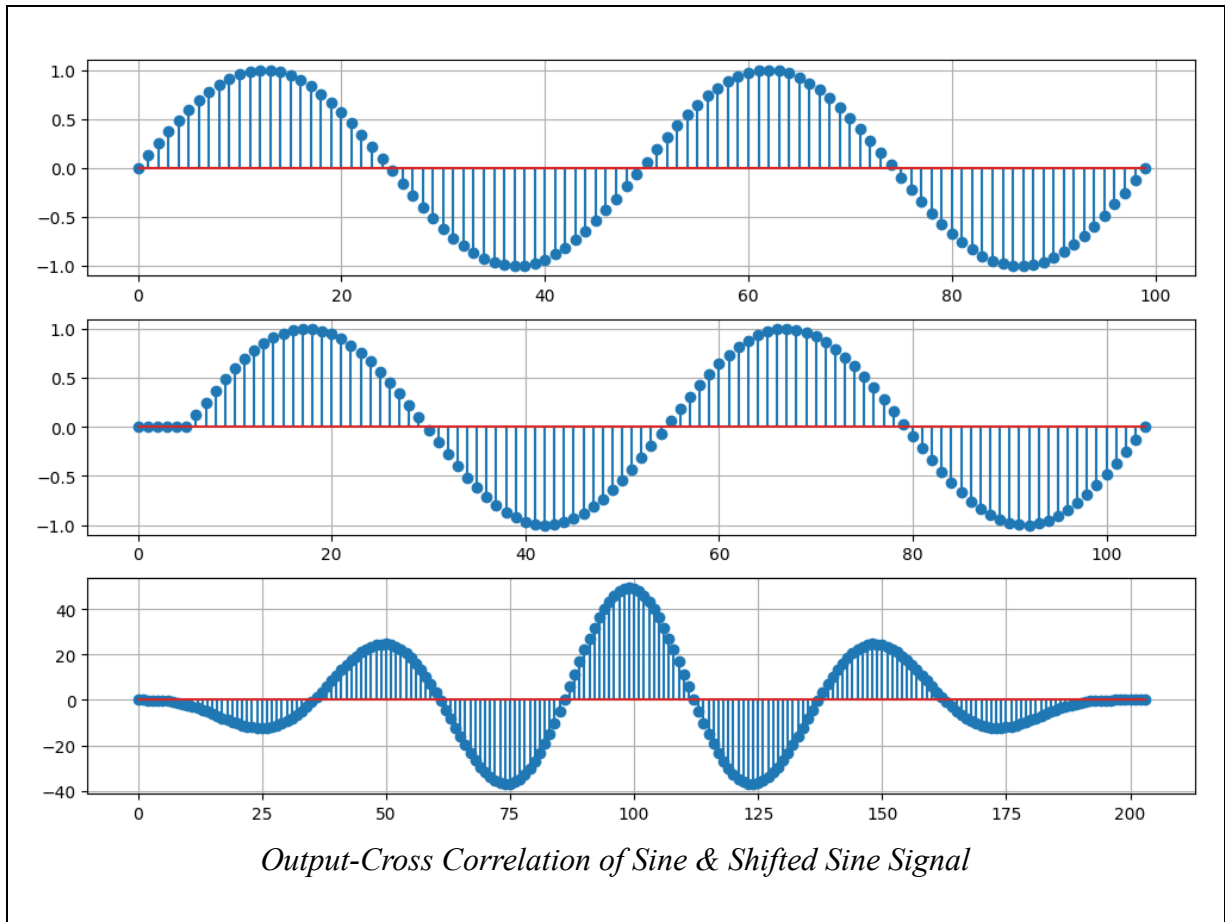
cross_corr=np.correlate(sinx,shifted_sign,'full')

plt.figure(figsize=(12,8))

plt.subplot(3,1,1)
plt.stem(sinx)
plt.grid(True)

plt.subplot(3,1,2)
plt.stem(shifted_sign)
plt.grid(True)

plt.subplot(3,1,3)
plt.stem(cross_corr)
plt.grid(True)
```



### Discussion:

- The cross-correlation of a sine wave and a shifted sine wave represent how much the second signal is delayed (or shifted) relative to the first one.
- Cross-correlating two sine waves, the result is not a flat line or a single spike. The result is another sinusoidal wave specifically, a cosine wave.

### c) Problem: Auto Correlation of Cosine Signal

#### Logical Solution:

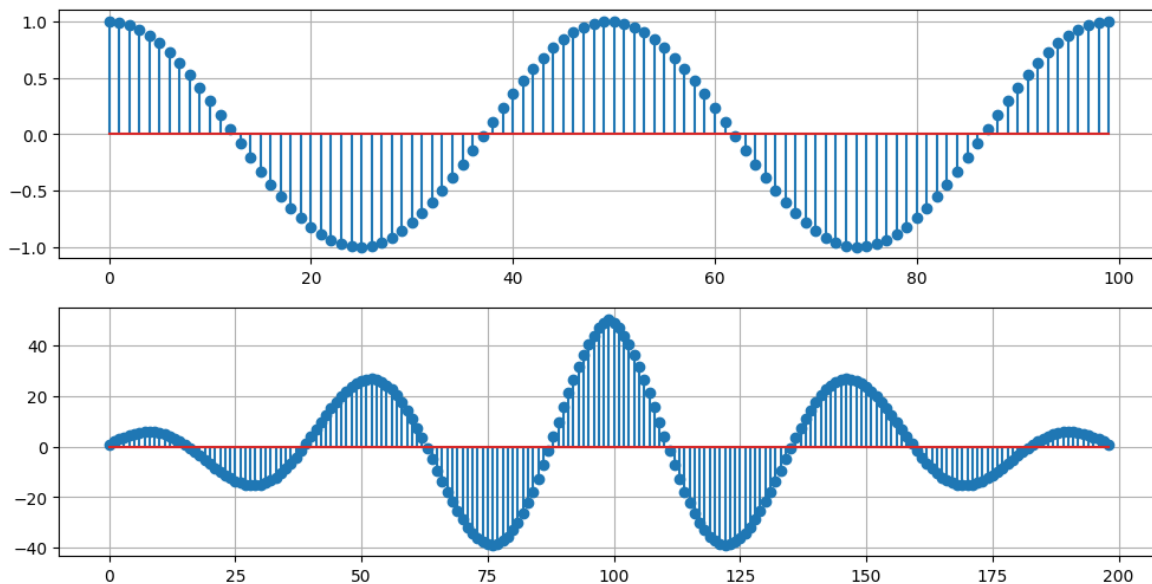
- $$r_{xx}(n) = \sum_{k=-\infty}^{\infty} x(k) * x(k - n)$$

#### Source Code:

```
import numpy as np
import matplotlib.pyplot as plt
```

```
f=1
t=np.linspace(-1,1,100)
cosx=np.cos(2*np.pi*f*t)
corr=np.correlate(cosx,cosx,'full')
```

```
plt.figure(figsize=(12,6))
plt.subplot(2,1,1)
plt.stem(cosx)
plt.grid(True)
plt.subplot(2,1,2)
plt.stem(corr)
plt.grid(True)
plt.show()
```



*Output- Auto Correlation of Cosine Signal*

### Discussion:

- A cosine signal and its autocorrelation result represents that it is a Power Signal as power is finite.
- The autocorrelation function confirms this classification based on the value at Zero Lag.
- The autocorrelation of a cosine signal measures how much the cosine wave resembles itself at different time delay.
- This tells that the self-similarity of the wave oscillates just like the wave itself does.